

SQL数据库工程师

知识库

从入门到精通职业技能培训教材

目 录

第1章 SQL基础与数据库概述
1.1 数据库基本概念与SQL语言简介
1.2 主流数据库管理系统对比
第2章 数据定义语言DDL
2.1 创建与管理数据库
2.2 创建与管理数据表
第3章 数据查询语言DQL基础
3.1 SELECT基本语法与单表查询
3.2 条件查询与排序
第4章 数据查询语言DQL高级
4.1 聚合函数与分组查询
4.2 多表连接查询
第5章 数据操作语言DML
5.1 数据的插入、更新与删除
5.2 批量数据操作与事务控制
第6章 数据控制语言DCL与权限管理
6.1 用户管理与权限分配
6.2 角色管理与安全策略
第7章 索引、视图与存储过程
7.1 索引原理与优化
7.2 视图与存储过程
第8章 SQL性能优化与故障排查
8.1 SQL执行计划与性能分析
8.2 常见SQL错误排查与最佳实践

第1章 SQL基础与数据库概述

1.1 数据库基本概念与SQL语言简介

【知识点讲解】

数据库 (Database) 是按照数据结构来组织、存储和管理数据的仓库。数据库管理系统 (DBMS) 是用于创建和管理数据库使用二维表结构存储数据, 表与表之间通过主键和外键建立关联。

SQL (Structured Query Language, 结构化查询语言) 是关系型数据库的标准操作语言, 由IBM在1970年代开发。SQL语言具有“做什么”, 无需描述“怎么做”。SQL语句主要分为四大类: 数据定义语言 (DDL)、数据查询语言 (DQL)、数据操作语言 (DML) 和数据控制语言 (DCL)。

【操作步骤或工作流程】

1. 安装数据库管理系统 (以MySQL为例): 下载MySQL Community Server安装包, 运行安装向导, 选择Server only模式, 设置root用户密码。
2. 连接数据库: 打开命令行工具, 输入命令 `mysql -u root -p`, 输入密码后进入MySQL交互环境。
3. 查看现有数据库: 执行 `SHOW DATABASES;` 命令, 系统将列出所有已创建的数据库。
4. 查看数据库版本: 执行 `SELECT VERSION();` 命令, 确认当前运行的数据库版本信息。

【注意事项】

SQL语句不区分大小写, 但建议关键字使用大写, 表名和列名使用小写, 以提高可读性。

每条SQL语句必须以分号 (;) 结尾, 但在部分数据库 (如SQL Server) 中分号可省略。

字符串常量必须使用单引号包裹, 如 '张三', 不能使用双引号 (MySQL中双引号与单引号等价, 但标准SQL只认单引号)。

注释方式: 单行注释使用 `--` 或 `#`, 多行注释使用 `/* */`。

【常见问题】

Q1: SQL和NoSQL有什么区别?

A: SQL数据库采用结构化表结构, 支持ACID事务, 适合复杂查询和关系型数据; NoSQL采用键值对、文档、列族或图结构, 适合大规模分布式存储和快速读写。

Q2: 学习SQL需要先学编程语言吗?

A: 不需要。SQL是声明式语言, 语法接近自然语言, 即使没有编程基础也能快速上手。但了解基本的数据类型和逻辑概念会有帮助。

【实际案例】

【案例: 电商公司数据库选型】

某中型电商公司需要搭建商品管理系统。技术团队评估后选择MySQL 8.0作为数据库, 原因如下:

1. 开源免费, 社区活跃, 文档完善;
2. 支持InnoDB存储引擎, 提供事务支持和行级锁;
3. 与Java/Python等开发语言配合良好;
4. 主从复制机制可满足读写分离需求。

实施过程: 在CentOS服务器上安装MySQL 8.0.33, 创建数据库 `ecommerce`, 设置字符集为 `utf8mb4` 以支持emoji表情, 配置权限并启动服务, 最终通过Navicat工具进行可视化管理。

1.2 主流数据库管理系统对比

【知识点讲解】

目前市场上主流的关系型数据库包括MySQL、PostgreSQL、Oracle、SQL Server和SQLite等。MySQL由Oracle公司维护, 是最广泛应用于Web开发。PostgreSQL功能最为丰富, 支持JSON、数组、自定义类型等高级特性, 被誉为“开源版Oracle”。Oracle提供强大的并发处理、安全性和高可用性, 但授权费用高昂。SQL Server是微软推出的数据库产品, 与.NET生态深度集成。SQLite是嵌入式数据库, 零配置、单文件存储, 适合移动应用和小型工具。

【操作步骤或工作流程】

1. 评估业务需求: 确定数据量规模 (百万级/千万级/亿级)、并发访问量、事务复杂度、预算限制。
2. 对比技术特性: 查看各数据库对存储过程、触发器、分区表、全文索引、JSON支持等特性的支持程度。
3. 考虑生态兼容性: 检查与现有开发框架、ORM工具、监控系统的兼容性。
4. 进行性能压测: 使用sysbench或JMeter等工具模拟实际业务场景, 对比TPS和响应时间。
5. 制定迁移方案: 如果选择切换数据库, 需评估数据迁移成本、SQL语法差异和应用程序改造工作量。

【注意事项】

MySQL 8.0之前默认使用MyISAM引擎，不支持事务，生产环境必须显式指定InnoDB引擎。

PostgreSQL的SQL语法与MySQL存在差异，如LIMIT在PostgreSQL中写作LIMIT/OFFSET，而MySQL使用LIMIT m,n。

Oracle的PL/SQL是专有扩展，与标准SQL差异较大，迁移成本较高。

SQL Server的T-SQL使用GO作为批处理分隔符，这是其他数据库不支持的语法。

【常见问题】

Q1：小企业应该选择免费数据库还是商业数据库？

A：对于数据量小于千万级、并发低于1000的中小型企业，MySQL或PostgreSQL完全够用，且社区版免费。只有当需要高级功能时才考虑Oracle或SQL Server商业版。

Q2：不同数据库的SQL语句通用吗？

A：基础的SELECT/INSERT/UPDATE/DELETE语句高度通用，但高级特性如分页、日期函数、字符串函数、存储过程语法存在差异。实际开发中建议使用ORM框架（如MyBatis、Hibernate）来屏蔽底层差异。

【实际案例】

【案例：金融系统数据库迁移】

某城商行核心系统原使用Oracle 11g，年授权费用超过200万元。为降低成本，技术团队制定三年迁移计划：

1. 第一阶段：非核心系统（报表查询、日志分析）迁移至PostgreSQL，验证稳定性；
2. 第二阶段：中间业务系统（支付网关、清算系统）迁移，重写PL/SQL为PostgreSQL函数；
3. 第三阶段：核心账务系统分库分表后迁移至MySQL集群，使用ShardingSphere实现分布式事务。

迁移过程中遇到的主要问题：Oracle的ROWNUM分页需改为MySQL的LIMIT；Oracle的NVL函数需改为MySQL的IFNULL；日期格式函数差异导致部分报表计算错误。通过建立SQL语法对照表和自动化转换脚本，最终成功完成迁移，年IT成本降低60%。

第2章 数据定义语言DDL

2.1 创建与管理数据库

【知识点讲解】

DDL (Data Definition Language, 数据定义语言) 用于定义数据库的结构, 包括创建、修改和删除数据库对象。数据库是表。一个数据库服务器可以管理多个数据库。创建数据库时需要指定字符集和排序规则, 字符集决定支持哪些字符 (如utf8mb4)。排序规则决定字符串比较和排序的方式 (如utf8mb4_unicode_ci不区分大小写)。

【操作步骤或工作流程】

1. 创建数据库：

```
CREATE DATABASE IF NOT EXISTS shop_db
```

```
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_unicode_ci;
```

2. 查看数据库创建语句：

```
SHOW CREATE DATABASE shop_db;
```

3. 切换当前数据库：

```
USE shop_db;
```

4. 修改数据库字符集：

```
ALTER DATABASE shop_db
```

```
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_unicode_ci;
```

5. 删除数据库 (谨慎操作)：

```
DROP DATABASE IF EXISTS shop_db;
```

6. 查看当前连接的数据库：

```
SELECT DATABASE();
```

【注意事项】

创建数据库前应先确认该名称是否已存在, 使用IF NOT EXISTS避免报错。

生产环境严禁直接执行DROP DATABASE, 必须先进行完整备份。

字符集推荐统一使用utf8mb4, 它比utf8支持更完整的Unicode字符 (包括4字节字符)。

数据库名称应避免使用SQL关键字 (如database、table、select等) 和特殊字符。

在Linux系统中, 数据库名称区分大小写; 在Windows系统中不区分。

【常见问题】

Q1：为什么推荐utf8mb4而不是utf8？

A：MySQL早期的utf8实际上是utf8mb3, 只支持3字节UTF-8字符, 无法存储emoji (如) 和部分生僻汉字。utf8mb4支持完整是MySQL 8.0的默认字符集。

Q2：如何查看数据库占用的磁盘空间？

A：在MySQL中执行 SELECT table_schema, SUM(data_length+index_length)/1024/1024 AS size_mb FROM information_schema.tables GROUP BY table_schema; 可查看各数据库大小。

【实际案例】

【案例：跨国电商多语言数据库搭建】

某跨境电商公司需要同时支持中文、英文、日文、阿拉伯文和泰文。数据库架构师设计如下方案：

1. 创建主数据库 `ecommerce_global`，字符集设为 `utf8mb4`；
2. 为不同地区创建独立数据库：`ecommerce_cn`、`ecommerce_jp`、`ecommerce_us`；
3. 每个地区数据库使用适合当地语言的排序规则：
 - 中文：`utf8mb4_unicode_ci`
 - 日文：`utf8mb4_japanese_ci`
 - 阿拉伯文：`utf8mb4_unicode_ci`
4. 通过数据库链接（`FEDERATED`引擎）实现跨库查询。

实施中发现阿拉伯文从右向左显示需要前端配合RTL布局，数据库层面只需确保字符集正确存储即可。最终所有语言商品名称、描述、评

2.2 创建与管理数据表

【知识点讲解】

数据表（Table）是数据库中存储数据的基本单位，由行（Row/Record）和列（Column/Field）组成。创建表时需要定义列（非空、唯一等）。主键（PRIMARY KEY）用于唯一标识每一行记录，一个表只能有一个主键，且主键列不允许为空。外键（FOREIGN KEY）确保引用完整性。

【操作步骤或工作流程】

1. 创建员工表：

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    emp_name VARCHAR(50) NOT NULL,  
    gender CHAR(1) DEFAULT 'M',  
    birth_date DATE,  
    dept_id INT,  
    salary DECIMAL(10,2),  
    email VARCHAR(100) UNIQUE,  
    hire_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
) ENGINE=InnoDB;
```

2. 查看表结构：

```
DESC employees;  
或 SHOW CREATE TABLE employees;
```

3. 修改表结构（添加列）：

```
ALTER TABLE employees ADD phone VARCHAR(20) AFTER email;
```

4. 修改列数据类型：

```
ALTER TABLE employees MODIFY salary DECIMAL(12,2);
```

5. 删除列：

```
ALTER TABLE employees DROP COLUMN phone;
```

6. 重命名表：

```
RENAME TABLE employees TO staff;
```

7. 删除表：

```
DROP TABLE IF EXISTS staff;
```

【注意事项】

设计表结构时应遵循数据库范式（至少满足第三范式），减少数据冗余。

VARCHAR适合存储长度变化大的字符串，CHAR适合固定长度（如身份证号、手机号）。

DECIMAL用于精确计算（如金额），FLOAT/DOUBLE用于科学计算但不精确。

主键建议使用自增整数（AUTO_INCREMENT），避免使用业务字段（如手机号可能变更）。

外键约束会影响插入和删除性能，高并发场景下可考虑在应用层保证数据一致性。

【常见问题】

Q1：INT(10)和INT(11)有什么区别？

A：括号中的数字只是显示宽度，不影响存储范围。INT始终占用4字节，范围是-2147483648到2147483647。显示宽度在

Q2：为什么删除表后自增ID不会从1开始？

A：自增计数器存储在内存中，TRUNCATE TABLE会重置计数器，但DELETE不会。如需重置，可执行 ALTER TABLE table_name AUTO_INCREMENT = 1;

【实际案例】

【案例：人力资源管理系统表设计】

某企业HR系统需要管理员工、部门、职位、薪资等信息。数据库设计如下：

1. departments表：

```
CREATE TABLE departments (  
  dept_id INT PRIMARY KEY AUTO_INCREMENT,  
  dept_name VARCHAR(50) NOT NULL,  
  location VARCHAR(100),  
  manager_id INT  
);
```

2. employees表（含外键）：

```
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY AUTO_INCREMENT,  
  emp_name VARCHAR(50) NOT NULL,  
  dept_id INT,  
  job_title VARCHAR(50),  
  salary DECIMAL(10,2),  
  hire_date DATE,  
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
  ON DELETE SET NULL ON UPDATE CASCADE  
);
```

关键设计决策：

- 外键设置ON DELETE SET NULL，当部门被删除时，该部门员工记录保留但dept_id变为NULL，避免误删员工数据。
- 设置ON UPDATE CASCADE，当部门ID变更时，自动同步更新员工表中的dept_id。
- 薪资使用DECIMAL(10,2)，确保精确到分，避免浮点数精度问题导致薪资计算错误。

实施后HR部门反馈：通过外键约束有效防止了录入不存在的部门编号，数据质量显著提升。

第3章 数据查询语言DQL基础

3.1 SELECT基本语法与单表查询

【知识点讲解】

DQL（Data Query Language，数据查询语言）是SQL中使用频率最高的部分，核心语句是SELECT。SELECT语句用于从数据库中SELECT 列名 FROM 表名 WHERE 条件。查询结果以虚拟表（结果集）的形式返回，不会修改原始数据。可以使用星号（*）使用DISTINCT关键字可以去除重复记录。AS关键字用于为列或表设置别名。

【操作步骤或工作流程】

1. 查询所有列：

```
SELECT * FROM employees;
```

2. 查询指定列：

```
SELECT emp_id, emp_name, salary FROM employees;
```

3. 使用别名：

```
SELECT emp_name AS 姓名, salary AS 月薪 FROM employees;
```

4. 去除重复值：

```
SELECT DISTINCT dept_id FROM employees;
```

5. 查询并计算：

```
SELECT emp_name, salary, salary * 12 AS annual_salary FROM employees;
```

6. 限制返回行数（MySQL）：

```
SELECT * FROM employees LIMIT 10;
```

7. 限制返回行数（SQL Server）：

```
SELECT TOP 10 * FROM employees;
```

8. 限制返回行数（Oracle）：

```
SELECT * FROM employees WHERE ROWNUM <= 10;
```

【注意事项】

SELECT * 虽然方便，但在生产环境中应避免使用，因为会增加网络传输量和内存消耗，且表结构变更时可能导致应用程序别名仅在当前查询中有效，不会修改表的实际列名。

进行数值计算时，注意NULL值参与运算的结果仍为NULL，应使用IFNULL/COALESCE处理。

LIMIT子句在MySQL中位置在最后，在Oracle中需配合ROWNUM或FETCH FIRST实现。

【常见问题】

Q1：SELECT 1 FROM table 是什么意思？

A：这种写法用于快速判断表中是否有记录，或作为子查询存在性检查。它不返回实际列数据，只返回常量1，性能比SELECT *

Q2：为什么查询结果列顺序与表定义不一致？

A：使用SELECT *时，列顺序始终与表定义一致。如果手动指定列名，则按指定顺序返回。建议始终显式指定列名。

【实际案例】

【案例：销售报表数据提取】

某零售公司需要每日生成销售日报，提取昨日所有销售记录。表结构：sales(sale_id, product_id, quantity, unit_price,

sale_date, store_id)。

查询语句：

```
SELECT
product_id AS 商品编号,
quantity AS 销售数量,
unit_price AS 单价,
quantity * unit_price AS 销售额
FROM sales
WHERE sale_date = CURDATE() - INTERVAL 1 DAY
ORDER BY 销售额 DESC
LIMIT 100;
```

优化点：

1. 显式指定所需列，避免传输无关数据；
2. 使用表达式计算销售额，减少应用层计算；
3. 按销售额降序排列，快速定位Top100商品；
4. 在sale_date列建立索引，将查询时间从3秒降至0.1秒。

实施后日报生成时间从5分钟缩短至30秒，系统负载降低40%。

3.2 条件查询与排序

【知识点讲解】

条件查询通过WHERE子句筛选满足特定条件的记录。WHERE支持多种运算符：比较运算符（=、<>、>、<、>=、<=）、逻辑运算符（AND、OR、NOT）、范围运算符（BETWEEN...AND）、集合运算符（IN、NOT IN）、模糊匹配（LIKE）和空值判断（IS NULL、IS NOT NULL）。BY子句，可以按一个或多个列升序（ASC，默认）或降序（DESC）排列。

【操作步骤或工作流程】

1. 精确条件查询：

```
SELECT * FROM employees WHERE dept_id = 3;
```

2. 多条件组合：

```
SELECT * FROM employees
WHERE salary > 5000 AND dept_id = 3;
```

3. 范围查询：

```
SELECT * FROM employees
WHERE salary BETWEEN 3000 AND 8000;
```

4. 集合查询：

```
SELECT * FROM employees
WHERE dept_id IN (1, 3, 5);
```

5. 模糊查询（%匹配任意字符，_匹配单个字符）：

```
SELECT * FROM employees WHERE emp_name LIKE '张%';
SELECT * FROM employees WHERE emp_name LIKE '_三';
```

6. 空值判断：

```
SELECT * FROM employees WHERE email IS NULL;
```

7. 排序：

```
SELECT * FROM employees
ORDER BY salary DESC, hire_date ASC;
```

8. 分页查询：

```
SELECT * FROM employees
ORDER BY emp_id
LIMIT 10 OFFSET 20;
```

【注意事项】

WHERE子句中不能使用聚合函数，需要使用HAVING（在GROUP BY后使用）。

NULL值不能使用=或<>比较，必须使用IS NULL或IS NOT NULL。

LIKE模糊查询以%开头（如'%张'）将无法使用索引，会导致全表扫描。

AND优先级高于OR，复杂条件建议用括号明确优先级。

ORDER BY多个列时，先按第一个列排序，相同值再按第二个列排序。

【常见问题】

Q1：WHERE 1=1 有什么用？

A：这是动态SQL拼接时的技巧。当需要动态添加AND条件时，以WHERE 1=1开头，后续条件统一用AND拼接，避免判断是否可能影响查询优化器。

Q2：LIKE和REGEXP有什么区别？

A：LIKE使用简单的通配符（%和_），REGEXP（或RLIKE）支持正则表达式，功能更强大但性能较差。MySQL中REGEXP不区分大小写，如需区分可使用BINARY关键字。

【实际案例】

【案例：会员筛选营销活动】

某电商平台要针对高价值会员进行精准营销，筛选条件：

- 注册时间超过1年
- 近3个月有购买记录
- 累计消费金额超过5000元
- 非黑名单用户

SQL实现：

```
SELECT
user_id,
username,
phone,
total_amount
FROM users u
JOIN orders o ON u.user_id = o.user_id
WHERE u.register_date <= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
AND u.is_blacklist = 0
AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 3 MONTH)
GROUP BY u.user_id
HAVING SUM(o.total_amount) > 5000
ORDER BY total_amount DESC
LIMIT 500;
```

营销效果：精准筛选出482名高价值会员，发送优惠券后复购率达到35%，远高于全量发送的5%复购率。

第4章 数据查询语言DQL高级

4.1 聚合函数与分组查询

【知识点讲解】

聚合函数对一组值进行计算并返回单个值，常用聚合函数包括：COUNT（计数）、SUM（求和）、AVG（平均值）、MAX（最大值）、MIN（最小值）。GROUP BY子句将数据按指定列分组，然后对每个组应用聚合函数。HAVING子句用于对分组后的结果进行筛选（类似于WHERE，但WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT。

【操作步骤或工作流程】

1. 统计员工总数：

```
SELECT COUNT(*) FROM employees;
```

2. 统计有邮箱的员工数：

```
SELECT COUNT(email) FROM employees;
```

3. 计算平均工资和最高工资：

```
SELECT AVG(salary) AS 平均工资, MAX(salary) AS 最高工资 FROM employees;
```

4. 按部门统计人数和平均工资：

```
SELECT dept_id, COUNT(*) AS 人数, AVG(salary) AS 平均工资  
FROM employees  
GROUP BY dept_id;
```

5. 筛选人数超过5人的部门：

```
SELECT dept_id, COUNT(*) AS 人数  
FROM employees  
GROUP BY dept_id  
HAVING COUNT(*) > 5;
```

6. 多字段分组统计：

```
SELECT dept_id, gender, COUNT(*) AS 人数, AVG(salary) AS 平均工资  
FROM employees  
GROUP BY dept_id, gender;
```

7. 使用WITH ROLLUP生成汇总行：

```
SELECT dept_id, COUNT(*) AS 人数, SUM(salary) AS 总工资  
FROM employees  
GROUP BY dept_id WITH ROLLUP;
```

【注意事项】

COUNT(*)统计所有行，COUNT(列名)统计该列非NULL的行数，COUNT(DISTINCT 列名)统计去重后的数量。

使用GROUP BY时，SELECT列表中只能出现分组列和聚合函数，其他列必须也出现在GROUP BY中（MySQL宽松模式下允许）。HAVING必须配合GROUP BY使用，且只能过滤聚合结果，不能替代WHERE过滤原始行。

聚合函数会忽略NULL值，但COUNT(*)不会忽略任何行。

【常见问题】

Q1：WHERE和HAVING有什么区别？

A：WHERE在分组前过滤原始数据行，HAVING在分组后过滤聚合结果。WHERE效率更高，应优先使用。例如筛选工资>5000的员工用WHERE，筛选平均工资>5000的部门用HAVING。

Q2：GROUP BY后为什么有些列显示不正确？

A：在标准SQL中，SELECT中非聚合列必须出现在GROUP BY中。MySQL的ONLY_FULL_GROUP_BY模式关闭时允许省略，但返回的值是该列的结果不确定。建议始终开启ONLY_FULL_GROUP_BY。

【实际案例】

【案例：月度销售统计分析】

某连锁超市需要按月统计各门店销售数据，要求：

1. 统计每月总销售额、总订单数、客单价（销售额/订单数）
2. 只显示销售额超过10万元的月份
3. 按销售额降序排列

SQL实现：

```
SELECT
DATE_FORMAT(order_date, '%Y-%m') AS 月份,
store_id AS 门店,
COUNT(DISTINCT order_id) AS 订单数,
SUM(total_amount) AS 销售额,
ROUND(SUM(total_amount) / COUNT(DISTINCT order_id), 2) AS 客单价
FROM orders
WHERE order_date >= '2024-01-01'
GROUP BY DATE_FORMAT(order_date, '%Y-%m'), store_id
HAVING SUM(total_amount) > 100000
ORDER BY 销售额 DESC;
```

分析结果：发现3月份华东区门店客单价异常偏低，进一步排查发现是促销活动期间大量小额订单导致。据此调整促销策略

4.2 多表连接查询

【知识点讲解】

多表连接查询用于从两个或多个表中获取相关数据。连接类型包括：内连接（INNER JOIN，只返回匹配的行）、左外连接（LEFT JOIN，返回左表所有行）、右外连接（RIGHT JOIN，返回右表所有行）、全外连接（FULL OUTER JOIN，返回两边所有行，MySQL不支持，可用UNION JOIN，笛卡尔积）。连接条件通常使用ON子句指定，也可以使用USING(列名)当两表有同名列时。

【操作步骤或工作流程】

1. 内连接查询员工及部门名称：

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

2. 左连接查询所有员工（包括未分配部门的）：

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

3. 多表连接（员工-部门-职位）：

```
SELECT e.emp_name, d.dept_name, j.job_title
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
JOIN jobs j ON e.job_id = j.job_id;
```

4. 自连接查询员工及其上级：

```
SELECT e.emp_name AS 员工, m.emp_name AS 上级
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

5. 使用USING简化（列名相同）：

```
SELECT * FROM employees e
JOIN departments d USING(dept_id);
```

6. 全外连接（MySQL用UNION模拟）：

```
SELECT e.emp_name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
```

UNION

```
SELECT e.emp_name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

【注意事项】

连接查询必须指定连接条件，否则会产生笛卡尔积（行数=表A行数×表B行数），导致性能灾难。

多表连接时，建议为每个表设置简短别名，减少SQL长度并提高可读性。

左连接时，右表不匹配列返回NULL，应用层需做好NULL处理。

连接列上应建立索引，否则大表连接会产生严重的性能问题。

超过3个表的复杂连接应谨慎设计，考虑拆分为多个简单查询或在应用层组装。

【常见问题】

Q1：INNER JOIN和WHERE连接有什么区别？

A：功能上等价，但INNER JOIN是SQL-92标准语法，语义更清晰（连接条件在ON中，过滤条件在WHERE中）。旧式逗号连接a.id=b.id）已不推荐使用。

Q2：LEFT JOIN后WHERE过滤和ON过滤有什么区别？

A：ON中的条件在连接时过滤，不影响左表行数；WHERE中的条件在连接后过滤，会把左表中不符合条件的行也过滤掉。例如NULL用于查找没有匹配的记录。

【实际案例】

【案例：订单详情完整信息查询】

某B2B电商平台需要展示订单完整信息，涉及4张表：

- orders（订单主表）：order_id, customer_id, order_date, status
- customers（客户表）：customer_id, company_name, contact_name, region
- order_items（订单明细表）：item_id, order_id, product_id, quantity, price
- products（商品表）：product_id, product_name, category

查询语句：

```
SELECT
o.order_id AS 订单号,
c.company_name AS 客户名称,
c.region AS 地区,
p.product_name AS 商品,
p.category AS 品类,
oi.quantity AS 数量,
oi.price AS 单价,
oi.quantity * oi.price AS 小计,
o.order_date AS 下单日期
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
```

```
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.order_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
ORDER BY o.order_date DESC;
```

实施效果：通过合理的索引设计（orders表的order_date和customer_id，order_items表的order_id和product_id），4表连接查询在百万级数据量下响应时间稳定在200ms以内，满足实时报表需求。

第5章 数据操作语言DML

5.1 数据的插入、更新与删除

【知识点讲解】

DML (Data Manipulation Language, 数据操作语言) 用于对表中的数据进行增删改操作, 包括INSERT (插入)、UPDATE (更新) 和 DELETE (删除)。INSERT用于向表中添加新记录, 可以插入完整行或部分列。UPDATE用于修改已有记录的值, 必须配合WHERE子句使用, 同样必须配合WHERE子句, 否则将清空整张表 (但保留表结构)。TRUNCATE TABLE也能清空表, 但速度更快且不记录单行。

【操作步骤或工作流程】

1. 插入完整行 :

```
INSERT INTO employees (emp_id, emp_name, gender, salary, dept_id)
VALUES (101, '张三', 'M', 8000.00, 1);
```

2. 插入多行 :

```
INSERT INTO employees (emp_name, gender, salary, dept_id)
VALUES
('李四', 'F', 7500.00, 2),
('王五', 'M', 9000.00, 1),
('赵六', 'F', 6500.00, 3);
```

3. 插入查询结果 :

```
INSERT INTO employees_backup
SELECT * FROM employees WHERE dept_id = 1;
```

4. 更新指定记录 :

```
UPDATE employees
SET salary = salary * 1.1
WHERE dept_id = 1;
```

5. 更新多列 :

```
UPDATE employees
SET salary = 10000, dept_id = 2
WHERE emp_id = 101;
```

6. 删除指定记录 :

```
DELETE FROM employees WHERE emp_id = 101;
```

7. 删除所有记录 (保留表结构) :

```
DELETE FROM employees;
```

或 TRUNCATE TABLE employees;

8. 使用REPLACE (MySQL特有, 存在则更新, 不存在则插入) :

```
REPLACE INTO employees (emp_id, emp_name, salary)
```

```
VALUES (101, '张三', 12000);
```

【注意事项】

INSERT时如果省略列名, VALUES必须按表定义的顺序提供所有列的值。

具有AUTO_INCREMENT的列可以传入NULL或省略, 数据库会自动生成值。

UPDATE和DELETE必须加WHERE条件, 否则操作整张表, 这是生产环境最常见的人为事故原因。

TRUNCATE TABLE是DDL操作，不能回滚（在某些数据库中），且会重置自增计数器。
外键约束可能阻止DELETE操作，需先删除或更新子表记录，或设置ON DELETE CASCADE。

【常见问题】

Q1：DELETE和TRUNCATE有什么区别？

A：DELETE是DML操作，逐行删除，记录日志，可回滚，不重置自增计数器，触发器会触发；TRUNCATE是DDL操作，直接删除表数据，重置自增计数器，不触发触发器。

Q2：如何安全地执行UPDATE/DELETE？

A：执行前先写SELECT语句验证WHERE条件影响的行数，确认无误后再将SELECT改为UPDATE/DELETE。或使用LIMIT限制影响行数，或开启安全更新模式（MySQL的sql_safe_updates）。

【实际案例】

【案例：薪资调整批量操作】

某互联网公司每年进行薪资普调，规则如下：

1. 绩效A的员工涨薪15%
2. 绩效B的员工涨薪8%
3. 绩效C的员工不涨薪
4. 入职不满一年的员工不参与调薪

实施步骤：

1. 先备份数据：CREATE TABLE employees_backup AS SELECT * FROM employees;

2. 验证影响范围：

```
SELECT emp_id, emp_name, salary, performance, hire_date
FROM employees
```

```
WHERE DATEDIFF(CURDATE(), hire_date) >= 365
```

```
AND performance IN ('A', 'B');
```

3. 执行批量更新（分两次）：

```
UPDATE employees
```

```
SET salary = ROUND(salary * 1.15, 2)
```

```
WHERE performance = 'A'
```

```
AND DATEDIFF(CURDATE(), hire_date) >= 365;
```

```
UPDATE employees
```

```
SET salary = ROUND(salary * 1.08, 2)
```

```
WHERE performance = 'B'
```

```
AND DATEDIFF(CURDATE(), hire_date) >= 365;
```

4. 核对结果：

```
SELECT COUNT(*), SUM(salary - old_salary) AS 总调薪金额
```

```
FROM employees e
```

```
JOIN employees_backup b ON e.emp_id = b.emp_id
```

```
WHERE e.salary != b.salary;
```

结果：成功为312名员工调整薪资，总调薪金额186万元，无一人出错。备份机制确保即使操作失误也能快速恢复。

5.2 批量数据操作与事务控制

【知识点讲解】

事务（Transaction）是一组逻辑操作单元，要么全部执行成功，要么全部不执行，保证数据库从一个一致性状态转换到另一个一致性状态。事务具有原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）、持久性（Durability）。

MySQL默认开启自动提交（autocommit=1），每条SQL语句都是一个独立事务。通过START TRANSACTION或BEGIN可以显式开始事务，COMMIT提交事务，ROLLBACK回滚事务。SAVEPOINT用于在事务中设置保存点，允许回滚到指定点而非全部回滚。

【操作步骤或工作流程】

1. 关闭自动提交：


```
SET autocommit = 0;
```

2. 显式开启事务：

```
START TRANSACTION;
```

或 BEGIN;

3. 执行一系列DML操作：

```
INSERT INTO accounts (account_id, balance) VALUES ('A001', 10000);
INSERT INTO accounts (account_id, balance) VALUES ('B001', 5000);
UPDATE accounts SET balance = balance - 2000 WHERE account_id = 'A001';
UPDATE accounts SET balance = balance + 2000 WHERE account_id = 'B001';
```

4. 设置保存点：

```
SAVEPOINT sp1;
```

5. 提交事务：

```
COMMIT;
```

6. 回滚到保存点：

```
ROLLBACK TO sp1;
```

7. 回滚整个事务：

```
ROLLBACK;
```

8. 批量插入优化（关闭唯一性检查和外键检查）：

```
SET unique_checks = 0;
SET foreign_key_checks = 0;
```

-- 执行大量INSERT

```
SET unique_checks = 1;
SET foreign_key_checks = 1;
```

【注意事项】

事务应尽量短小，长时间持有锁会导致其他会话阻塞，甚至产生死锁。

DDL语句（如CREATE、ALTER、DROP）会隐式提交当前事务，不能在事务中混用DDL和DML。

事务隔离级别包括：READ UNCOMMITTED（读未提交）、READ COMMITTED（读已提交）、REPEATABLE READ（可重复读）、SERIALIZABLE（串行化）。级别越高，并发性能越差，数据一致性越好。

死锁发生时，数据库会自动选择一个事务进行回滚，应用层需要捕获异常并重试。

【常见问题】

Q1：什么情况下会出现脏读、不可重复读、幻读？

A：脏读：事务A读取了事务B未提交的数据；不可重复读：事务A两次读取同一行，期间事务B修改并提交；幻读：事务A两次读取MySQL的REPEATABLE READ通过MVCC和间隙锁解决了幻读问题。

Q2：批量插入如何优化性能？

A：1. 使用多值INSERT（VALUES后接多组值）；2. 关闭自动提交，手动批量提交；3. 临时关闭唯一性检查和外键检查；4. INFILE替代INSERT；5. 调整innodb_buffer_pool_size和innodb_log_file_size。

【实际案例】

【案例：银行转账系统事务设计】

某银行核心系统处理跨行转账，涉及三个步骤：扣减付款方余额、增加收款方余额、记录交易流水。任何一步失败都必须回

实现代码：

```
START TRANSACTION;
```

```
-- 检查付款方余额
```

```
SELECT balance INTO @bal FROM accounts  
WHERE account_id = '62220001' FOR UPDATE;
```

```
IF @bal < 5000 THEN
```

```
ROLLBACK;
```

```
SELECT '余额不足' AS result;
```

```
ELSE
```

```
-- 扣减付款方
```

```
UPDATE accounts SET balance = balance - 5000  
WHERE account_id = '62220001';
```

```
-- 增加收款方
```

```
UPDATE accounts SET balance = balance + 5000  
WHERE account_id = '62220002';
```

```
-- 记录流水
```

```
INSERT INTO transactions (tx_id, from_acc, to_acc, amount, tx_time)  
VALUES ('TX20240801001', '62220001', '62220002', 5000, NOW());
```

```
COMMIT;
```

```
SELECT '转账成功' AS result;
```

```
END IF;
```

关键设计：

1. 使用SELECT ... FOR UPDATE对付款方账户加排他锁，防止并发扣款导致超支；
2. 所有操作在一个事务内完成，确保原子性；
3. 事务超时设置为30秒，避免长时间锁等待；
4. 应用层实现重试机制，处理死锁异常。

上线后日均处理转账10万笔，零资金差错事故。

第6章 数据控制语言DCL与权限管理

6.1 用户管理与权限分配

【知识点讲解】

DCL（Data Control Language，数据控制语言）用于管理数据库访问权限和安全性，主要包括GRANT（授权）和REVOKE（撤销）。数据库用户管理是安全的第一道防线，需要遵循最小权限原则：每个用户只授予完成工作所必需的最小权限集合。MySQL的表权限和列权限。用户认证信息存储在mysql系统数据库的user表中。

【操作步骤或工作流程】

1. 创建新用户：

```
CREATE USER 'app_user'@'192.168.1.%'  
IDENTIFIED BY 'StrongP@sswOrd';
```

2. 查看所有用户：

```
SELECT user, host FROM mysql.user;
```

3. 授予数据库所有权限：

```
GRANT ALL PRIVILEGES ON shop_db.*  
TO 'app_user'@'192.168.1.%';
```

4. 授予特定权限：

```
GRANT SELECT, INSERT, UPDATE ON shop_db.orders  
TO 'order_writer'@'localhost';
```

5. 授予列级权限：

```
GRANT SELECT (emp_id, emp_name), UPDATE (salary)  
ON hr_db.employees  
TO 'hr_clerk'@'localhost';
```

6. 刷新权限（使授权立即生效）：

```
FLUSH PRIVILEGES;
```

7. 查看用户权限：

```
SHOW GRANTS FOR 'app_user'@'192.168.1.%';
```

8. 撤销权限：

```
REVOKE DELETE ON shop_db.* FROM 'app_user'@'192.168.1.%';
```

9. 修改用户密码：

```
ALTER USER 'app_user'@'192.168.1.%'  
IDENTIFIED BY 'NewP@sswOrd';
```

10. 删除用户：

```
DROP USER 'app_user'@'192.168.1.%';
```

【注意事项】

用户名格式为'用户名'@'主机'，主机可以是IP、网段（如192.168.1.%）、域名或localhost。 '%'表示任意主机。生产环境密码必须复杂（包含大小写字母、数字、特殊符号，长度 ≥ 12 ），并定期更换。

root用户应限制登录IP（如只允许localhost），禁止远程root登录。
使用SSL连接加密传输，防止密码和数据在传输过程中被窃听。
定期审计用户权限，及时清理离职员工账号和不再使用的测试账号。

【常见问题】

Q1：为什么授权后需要FLUSH PRIVILEGES？

A：MySQL启动时将权限表加载到内存，GRANT和REVOKE语句会自动刷新内存中的权限，但手动修改mysql.user表后必须执行FLUSH PRIVILEGES才能生效。
正常使用GRANT不需要手动刷新。

Q2：如何限制用户只能查看自己的数据？

A：可以通过视图（VIEW）实现行级安全，创建视图时加入WHERE user_id = CURRENT_USER()条件，然后只授予用户对视图的SELECT权限，不授予基表权限。

【实际案例】

【案例：多角色权限体系设计】

某SaaS电商平台需要为不同角色分配数据库权限：

角色设计：

1. 系统管理员（DBA）：拥有所有数据库的全部权限，仅限内网IP访问；
2. 应用服务账号：对业务库有SELECT/INSERT/UPDATE/DELETE权限，对日志库只有INSERT权限；
3. 数据分析师：对所有业务表有SELECT权限，对敏感字段（如手机号、身份证号）无权限；
4. 备份账号：只有SELECT和LOCK TABLES权限，用于执行物理备份；
5. 监控账号：只有PROCESS和REPLICATION CLIENT权限，用于监控工具连接。

实施SQL：

-- 应用服务账号

```
CREATE USER 'app_saas'@'10.0.0.0/24' IDENTIFIED BY 'App$2024#Secure';
GRANT SELECT, INSERT, UPDATE, DELETE ON saas_db.* TO 'app_saas'@'10.0.0.0/24';
GRANT INSERT ON saas_db.operation_logs TO 'app_saas'@'10.0.0.0/24';
```

-- 数据分析师（通过视图屏蔽敏感列）

```
CREATE VIEW customers_safe AS
SELECT customer_id, company_name, region, created_at
FROM customers;
GRANT SELECT ON saas_db.customers_safe TO 'analyst'@'10.0.0.0/24';
```

实施后通过权限审计发现3个冗余账号，及时清理，安全评分从B提升至A+。

6.2 角色管理与安全策略

【知识点讲解】

角色（Role）是一组权限的集合，可以将权限分配给角色，再将角色分配给用户，简化权限管理。MySQL 8.0正式支持角色，可用于实现基于岗位的权限管理、临时激活/禁用权限集。安全策略还包括密码策略（复杂度、过期时间）、登录失败锁定、审计日志等。网络隔离（VPC、防火墙）和定期漏洞扫描。

【操作步骤或工作流程】

1. 创建角色：

```
CREATE ROLE 'app_read', 'app_write', 'app_admin';
```

2. 为角色授权：

```
GRANT SELECT ON shop_db.* TO 'app_read';
GRANT SELECT, INSERT, UPDATE ON shop_db.* TO 'app_write';
GRANT ALL ON shop_db.* TO 'app_admin';
```

3. 将角色分配给用户：

```
GRANT 'app_read' TO 'analyst1'@'localhost';  
GRANT 'app_write' TO 'developer1'@'localhost';
```

4. 设置用户默认角色：

```
SET DEFAULT ROLE 'app_read' TO 'analyst1'@'localhost';
```

5. 激活角色（会话级别）：

```
SET ROLE 'app_write';
```

6. 查看当前激活的角色：

```
SELECT CURRENT_ROLE();
```

7. 设置密码策略（MySQL 8.0）：

```
SET GLOBAL validate_password.policy = STRONG;  
SET GLOBAL validate_password.length = 12;
```

8. 启用审计日志（MySQL Enterprise）：

```
INSTALL PLUGIN audit_log SONAME 'audit_log.so';
```

```
SET GLOBAL audit_log_policy = 'ALL';
```

【注意事项】

角色创建后默认未激活，需要SET ROLE激活或SET DEFAULT ROLE设置默认角色。

密码策略应平衡安全性和可用性，过于复杂可能导致用户将密码写在便利贴上。

审计日志会显著影响性能，生产环境应只审计关键表和敏感操作（如DROP、GRANT）。

定期使用mysql_secure_installation脚本进行安全检查，删除匿名用户和测试数据库。

数据库应部署在独立网段，通过防火墙限制只有应用服务器能访问数据库端口（默认3306）。

【常见问题】

Q1：角色和用户有什么区别？

A：用户是实际的数据库登录账号，角色是权限的容器。用户可以被授予角色，也可以直接被授权。角色不能被用于登录数据库。

Q2：如何防止SQL注入攻击？

A：1. 使用参数化查询（PreparedStatement），绝不拼接用户输入到SQL中；2. 最小权限原则，应用账号不授予DROP/GRANT；3. 使用ORM框架；4. 对用户输入进行白名单校验；5. 启用WAF（Web应用防火墙）。

【实际案例】

【案例：金融系统数据库安全加固】

某证券公司数据库安全整改项目，要求达到等保三级标准：

整改措施：

1. 账号整改：

- 删除所有共享账号，实现一人一号；
- root账号改为只允许本地登录，密码16位随机强密码；
- 创建dbbackup专用备份账号，仅授予SELECT和LOCK TABLES。

2. 角色体系：

```
CREATE ROLE 'trader_read', 'trader_write', 'risk_admin';  
GRANT SELECT ON trading_db.* TO 'trader_read';  
GRANT SELECT, INSERT, UPDATE ON trading_db.orders TO 'trader_write';  
GRANT ALL ON trading_db.* TO 'risk_admin';
```

3. 密码策略：

SET GLOBAL validate_password.policy = STRONG;

SET GLOBAL validate_password.length = 16;

SET GLOBAL default_password_lifetime = 90; -- 90天强制更换

4. 网络隔离：

- 数据库部署在独立VLAN，仅开放3306给应用服务器；
- 管理操作必须通过堡垒机（Jump Server）进行；
- 启用SSL连接，证书由内部CA签发。

5. 审计日志：

- 记录所有DDL操作和GRANT/REVOKE；
- 记录对敏感表（customer_assets）的所有访问；
- 日志实时同步至SIEM系统。

整改后通过等保三级测评，全年未发生数据库安全事件。

第7章 索引、视图与存储过程

7.1 索引原理与优化

【知识点讲解】

索引 (Index) 是数据库中用于加速数据检索的数据结构，类似于书籍的目录。没有索引时，数据库需要进行全表扫描 (Full Table Scan)，时间复杂度为 $O(n)$ 。使用索引后，数据库可以直接定位到符合条件的记录，时间复杂度降为 $O(\log n)$ 。MySQL的InnoDB引擎支持主索引、唯一索引、普通索引、组合索引和全文索引。索引虽然能加速查询，但会降低INSERT/UPDATE/DELETE的速度（需要维护索引）。

【操作步骤或工作流程】

1. 创建普通索引：

```
CREATE INDEX idx_emp_name ON employees(emp_name);
```

2. 创建唯一索引：

```
CREATE UNIQUE INDEX idx_email ON employees(email);
```

3. 创建组合索引：

```
CREATE INDEX idx_dept_salary ON employees(dept_id, salary);
```

4. 在创建表时定义索引：

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    category_id INT,  
    price DECIMAL(10,2),  
    INDEX idx_category (category_id),  
    INDEX idx_name_price (product_name, price)  
);
```

5. 查看表上的索引：

```
SHOW INDEX FROM employees;
```

6. 分析查询是否使用索引：

```
EXPLAIN SELECT * FROM employees WHERE emp_name = '张三';
```

7. 删除索引：

```
DROP INDEX idx_emp_name ON employees;
```

8. 重建索引（优化碎片）：

```
OPTIMIZE TABLE employees;
```

【注意事项】

索引列应出现在WHERE、JOIN、ORDER BY和GROUP BY子句中，SELECT列表中的列不需要索引。组合索引遵循最左前缀原则：索引(a,b,c)可以支持a、ab、abc的查询，但不能支持b或bc的查询。不要对低基数列（如性别、状态只有几个值）创建单独索引，索引选择性差，优化器可能放弃使用。频繁更新的列不适合建索引，维护索引的成本可能超过查询收益。索引不是越多越好，一般建议单表索引数量不超过5个，组合索引列数不超过3个。

【常见问题】

Q1：EXPLAIN结果中type列的含义？

A：type表示访问类型，从优到差依次为：system > const > eq_ref > ref > range > index > ALL。ALL表示全表扫描，index表示全索引扫描，range表示索引范围扫描，ref表示非唯一索引等值查询，eq_ref表示主键或唯一索引等值查询。

Q2：为什么有时索引没有被使用？

A：常见原因：1. 对索引列使用函数或运算（如WHERE YEAR(date_col)=2024）；2. 类型隐式转换（字符串传入数字）；3. 前缀模糊匹配；4. 数据量太小，优化器认为全表扫描更快；5. 索引列区分度太低；6. 统计信息过期，执行ANALYZE TABLE更新统计信息。

【实际案例】

【案例：电商订单查询优化】

某电商平台订单表orders有500万条记录，查询慢问题突出。

优化前：

```
SELECT * FROM orders
WHERE customer_id = 12345
AND order_date >= '2024-01-01'
AND status = 'completed'
ORDER BY order_date DESC
LIMIT 20;
-- 执行时间：8.5秒
```

问题诊断（EXPLAIN）：

- type: ALL（全表扫描）
- rows: 5000000
- Extra: Using where; Using filesort

优化方案：

1. 创建组合索引：

```
CREATE INDEX idx_cust_date_status
ON orders(customer_id, order_date, status);
```

2. 调整查询（避免SELECT *，只取所需列）：

```
SELECT order_id, total_amount, order_date, status
FROM orders
WHERE customer_id = 12345
AND order_date >= '2024-01-01'
AND status = 'completed'
ORDER BY order_date DESC
LIMIT 20;
```

优化后：

- type: range
- rows: 156
- Extra: Using index condition
- 执行时间：0.02秒

效果：查询性能提升400倍，数据库CPU使用率从85%降至25%。

7.2 视图与存储过程

【知识点讲解】

视图（View）是基于SQL查询结果的虚拟表，不存储实际数据，只保存查询定义。视图可以简化复杂查询、隐藏敏感数据（如敏感用户信息）。存储过程（Stored Procedure）是预编译的SQL语句集合，存储在数据库服务器端，客户端通过调用过程名执行。存储过程可以接收参数、返回结果。

WHILE等)。触发器 (Trigger) 是在特定事件 (INSERT/UPDATE/DELETE) 发生时自动执行的存储过程。

【操作步骤或工作流程】

1. 创建视图：

```
CREATE VIEW v_employee_details AS
SELECT e.emp_id, e.emp_name, d.dept_name, e.salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

2. 查询视图：

```
SELECT * FROM v_employee_details WHERE salary > 5000;
```

3. 创建存储过程：

```
DELIMITER //
CREATE PROCEDURE sp_get_employee_by_dept(IN dept_id INT)
BEGIN
    SELECT emp_id, emp_name, salary
    FROM employees
    WHERE dept_id = dept_id;
END //
DELIMITER ;
```

4. 调用存储过程：

```
CALL sp_get_employee_by_dept(1);
```

5. 创建带输出参数的存储过程：

```
DELIMITER //
CREATE PROCEDURE sp_get_salary_stats(
```

```
IN p_dept_id INT,
OUT p_avg_salary DECIMAL(10,2),
OUT p_max_salary DECIMAL(10,2)
)
```

```
BEGIN
    SELECT AVG(salary), MAX(salary)
    INTO p_avg_salary, p_max_salary
    FROM employees
    WHERE dept_id = p_dept_id;
END //
DELIMITER ;
```

6. 创建触发器 (记录薪资变更历史)：

```
DELIMITER //
CREATE TRIGGER trg_salary_change
```

```
AFTER UPDATE ON employees
```

```

FOR EACH ROW
BEGIN
    IF OLD.salary != NEW.salary THEN
        INSERT INTO salary_history (emp_id, old_salary, new_salary, change_date)
        VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
    END IF;
END //
DELIMITER ;

```

7. 删除视图/存储过程/触发器：

```

DROP VIEW IF EXISTS v_employee_details;
DROP PROCEDURE IF EXISTS sp_get_employee_by_dept;
DROP TRIGGER IF EXISTS trg_salary_change;

```

【注意事项】

视图中的查询如果涉及多表连接或聚合函数，可能无法直接进行INSERT/UPDATE/DELETE操作。
 存储过程在数据库端执行，减少网络传输，但会增加数据库服务器的计算压力，复杂业务逻辑建议在应用层实现。
 触发器是“隐形”的代码，过度使用会导致数据流难以追踪，调试困难，建议仅在审计、日志等场景使用。
 MySQL存储过程中使用DECLARE声明变量，使用SET赋值，使用SELECT ... INTO给变量赋值。
 创建存储过程和触发器前需先修改分隔符（DELIMITER），避免SQL语句中的分号提前结束命令。

【常见问题】

Q1：视图和表有什么区别？

A：表存储实际数据，视图只存储查询定义。对视图的查询最终会被数据库转换为对基表的查询。视图不占用数据存储空间
 视图可以限制用户访问基表的特定行或列。

Q2：存储过程和函数有什么区别？

A：存储过程用CALL调用，可以有多个输入输出参数，不返回值（但可以通过OUT参数返回）；函数用SELECT调用，必须有
 可以在SQL语句中直接使用（如SELECT my_func(col) FROM table）。

【实际案例】

【案例：销售报表视图与自动化统计】

某零售企业需要每日生成多维度销售报表，涉及复杂的聚合和多表连接。

方案设计：

1. 创建基础视图（隐藏复杂连接）：

```

CREATE VIEW v_daily_sales AS
SELECT
    DATE_FORMAT(o.order_date, '%Y-%m-%d') AS sale_date,
    s.store_name,
    c.category_name,
    SUM(oi.quantity) AS total_qty,
    SUM(oi.quantity * oi.price) AS total_amount,
    COUNT(DISTINCT o.order_id) AS order_count
FROM orders o
JOIN stores s ON o.store_id = s.store_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m-%d'), s.store_name, c.category_name;

```

2. 创建存储过程生成日报：

```

DELIMITER //
CREATE PROCEDURE sp_generate_daily_report(IN report_date DATE)

```

```

BEGIN
DECLARE done INT DEFAULT 0;
DECLARE v_store VARCHAR(50);
DECLARE v_amount DECIMAL(12,2);

-- 游标遍历各门店
DECLARE cur CURSOR FOR
SELECT store_name, SUM(total_amount)
FROM v_daily_sales
WHERE sale_date = report_date
GROUP BY store_name;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

OPEN cur;
read_loop: LOOP
FETCH cur INTO v_store, v_amount;
IF done THEN LEAVE read_loop; END IF;
INSERT INTO daily_reports (report_date, store_name, amount)
VALUES (report_date, v_store, v_amount);
END LOOP;
CLOSE cur;
END //
DELIMITER ;

```

3. 创建触发器自动记录异常大额订单：

```

CREATE TRIGGER trg_large_order_alert
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
IF NEW.total_amount > 50000 THEN
INSERT INTO alerts (alert_type, content, alert_time)
VALUES ('LARGE_ORDER',
CONCAT('大额订单：', NEW.order_id, ' 金额：', NEW.total_amount),
NOW());
END IF;
END;

```

实施效果：业务人员直接查询视图即可获取报表数据，无需理解底层表结构；日报生成从手动操作变为定时调用存储过程；大额订单触发

第8章 SQL性能优化与故障排查

8.1 SQL执行计划与性能分析

【知识点讲解】

SQL性能优化是数据库工程师的核心技能。优化流程通常为：发现问题（慢查询日志、监报告警）→ 定位问题（EXPLAIN分析）→ 验证效果（对比执行时间和资源消耗）。执行计划（Execution Plan）是数据库优化器生成的查询执行方案，通过EXPLAIN的select_type（查询类型）、type（访问类型）、key（使用的索引）、rows（扫描行数）、Extra（额外信息）。慢查询日志记录执行时间超过阈值的SQL，是发现性能瓶颈的重要工具。

【操作步骤或工作流程】

1. 开启慢查询日志（MySQL）：

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 2; -- 超过2秒记录
SET GLOBAL log_queries_not_using_indexes = 'ON';
```

2. 查看慢查询日志位置：

```
SHOW VARIABLES LIKE 'slow_query_log_file';
```

3. 使用EXPLAIN分析查询：

```
EXPLAIN SELECT * FROM orders
WHERE customer_id = 1001
ORDER BY order_date DESC;
```

4. 使用EXPLAIN ANALYZE（MySQL 8.0.18+）获取实际执行时间：

```
EXPLAIN ANALYZE SELECT * FROM orders
WHERE customer_id = 1001;
```

5. 查看当前正在执行的SQL：

```
SHOW PROCESSLIST;
或 SELECT * FROM information_schema.processlist
WHERE command != 'Sleep';
```

6. 查看表统计信息：

```
SHOW TABLE STATUS LIKE 'orders';
ANALYZE TABLE orders;
```

7. 使用性能模式（Performance Schema）分析：

```
SELECT * FROM performance_schema.events_statements_summary_by_digest
ORDER BY COUNT_STAR DESC LIMIT 10;
```

8. 使用SHOW PROFILE分析SQL各阶段耗时：

```
SET profiling = 1;
SELECT * FROM orders WHERE order_id = 10001;
SHOW PROFILES;
```

【注意事项】

慢查询日志在生产环境应谨慎开启，大量慢查询会导致日志文件迅速膨胀，建议配合日志轮转工具（如logrotate）。EXPLAIN显示的是预估执行计划，可能与实际执行有差异，MySQL 8.0.18+的EXPLAIN ANALYZE显示实际执行统计。优化器可能因统计信息过时而选择错误的执行计划，定期执行ANALYZE TABLE更新统计信息。

不要仅关注执行时间，还应关注锁等待时间、IO次数、临时表使用等深层指标。

使用pt-query-digest (Percona Toolkit) 工具可以高效分析慢查询日志，生成优化建议报告。

【常见问题】

Q1：Using filesort是什么意思？

A：表示MySQL需要额外的排序操作，通常是因为ORDER BY的列没有合适的索引。如果数据量大，filesort会使用磁盘临时表。BY列创建索引，或调整查询避免排序。

Q2：Using temporary是什么意思？

A：表示查询需要创建临时表来存储中间结果，常见于GROUP BY、DISTINCT、UNION等操作。如果临时表数据量超过tmp_table_size，严重影响性能。应尽量简化查询，确保GROUP BY列有索引。

【实际案例】

【案例：慢查询优化实战】

某物流系统订单追踪查询缓慢，高峰期CPU飙升至95%。

问题SQL：

```
SELECT o.order_id, o.customer_name, o.create_time,  
l.location, l.status, l.update_time  
FROM orders o  
LEFT JOIN logistics l ON o.order_id = l.order_id  
WHERE o.create_time >= DATE_SUB(NOW(), INTERVAL 7 DAY)  
AND (o.customer_name LIKE '%张三%' OR o.order_id LIKE '%10086%')  
ORDER BY o.create_time DESC  
LIMIT 50;
```

诊断过程：

1. EXPLAIN显示：type=ALL（全表扫描），rows=280万，Extra=Using where; Using filesort
2. 慢查询日志显示平均执行时间：12.3秒
3. SHOW PROCESSLIST显示大量查询处于"Sending data"状态

优化方案：

1. 去除前置模糊匹配（LIKE '%xxx%'无法使用索引），改为全文索引：
CREATE FULLTEXT INDEX idx_fulltext ON orders(customer_name, order_id);
查询改为：WHERE MATCH(customer_name, order_id) AGAINST('张三 10086')

2. 为create_time创建索引：

```
CREATE INDEX idx_create_time ON orders(create_time);
```

3. 优化SQL结构，避免不必要的LEFT JOIN：

```
SELECT o.order_id, o.customer_name, o.create_time,  
(SELECT location FROM logistics WHERE order_id = o.order_id ORDER BY update_time DESC  
LIMIT 1) AS location  
FROM orders o  
WHERE MATCH(customer_name, order_id) AGAINST('张三 10086')  
AND o.create_time >= DATE_SUB(NOW(), INTERVAL 7 DAY)  
ORDER BY o.create_time DESC  
LIMIT 50;
```

优化结果：

- 执行时间从12.3秒降至0.15秒
- CPU使用率从95%降至35%
- 慢查询日志中该SQL消失

- 用户投诉量下降90%

8.2 常见SQL错误排查与最佳实践

【知识点讲解】

SQL开发和运维过程中会遇到各类错误，掌握常见错误的排查方法是数据库工程师的必备技能。常见错误类型包括：语法错误（缺少权限）、数据错误（如主键冲突、外键约束失败、数据类型不匹配）、连接错误（如连接超时、Too many connections）。遵循SQL编写最佳实践可以显著减少错误发生概率，提高代码质量和可维护性。

【操作步骤或工作流程】

1. 语法错误排查：

- 检查关键字拼写（如SELET→SELECT，FRM→FROM）

- 检查括号是否成对闭合
- 检查字符串引号是否成对
- 检查逗号、分号位置

2. 权限错误排查：

```
SHOW GRANTS FOR CURRENT_USER();
```

-- 确认是否有对应数据库/表的权限

3. 主键冲突处理：

```
INSERT INTO users (id, name) VALUES (1, '张三')
ON DUPLICATE KEY UPDATE name = '张三';
```

-- 或使用REPLACE INTO

4. 外键约束错误排查：

```
SET foreign_key_checks = 0; -- 临时禁用（仅用于数据导入）
```

-- 导入完成后：

```
SET foreign_key_checks = 1;
```

5. 连接数过多处理：

```
SHOW VARIABLES LIKE 'max_connections';
SET GLOBAL max_connections = 500;
```

-- 查看连接来源：

```
SELECT user, host, COUNT(*) FROM information_schema.processlist
GROUP BY user, host;
```

6. 死锁排查：

```
SHOW ENGINE INNODB STATUS; -- 查看最近一次死锁信息
-- 或查询performance_schema：
SELECT * FROM information_schema.innodb_trx;
```

7. 数据类型不匹配处理：

-- 使用CAST转换类型

```
SELECT * FROM orders WHERE CAST(order_id AS CHAR) = '10001';
```

8. 锁等待超时处理：

```
SHOW VARIABLES LIKE 'innodb_lock_wait_timeout';  
SET GLOBAL innodb_lock_wait_timeout = 50;
```

【注意事项】

生产环境执行SQL前，务必在测试环境验证，并备份数据。

使用事务时确保有明确的COMMIT或ROLLBACK，避免长时间持有锁。

禁用外键检查仅用于数据迁移场景，正常业务必须保持外键约束开启。

增加max_connections不能解决根本问题，应检查是否有连接泄漏（未关闭的连接）。

死锁是正常现象，应用层应实现重试机制，但频繁死锁说明事务设计或索引有问题。

【常见问题】

Q1：ERROR 1064 (42000)是什么意思？

A：这是MySQL的语法错误代码，表示SQL语句有语法问题。错误信息通常会指出错误位置（如"near 'FROM'"），重点检查

Q2：ERROR 1205 (HY000): Lock wait timeout exceeded怎么办？

A：表示当前事务等待锁的时间超过了innodb_lock_wait_timeout设置。解决方法：1. 找出持有锁的会话（SHOW PROCESSLIST或information_schema.innodb_trx）；2. 终止长时间运行的会话（KILL id）；3. 优化事务逻辑，缩短事务持有检查是否有未提交的事务。

【实际案例】

【案例：生产环境数据修复】

某电商平台因程序Bug导致部分订单状态错误，需要将2024年7月1日至7月10日期间，status为'pending'且payment_status的订单状态更新为'processing'。

修复步骤：

1. 数据备份（必须先备份）：

```
CREATE TABLE orders_backup_20240715 AS  
SELECT * FROM orders  
WHERE order_date BETWEEN '2024-07-01' AND '2024-07-10';
```

2. 在测试环境验证SQL：

```
SELECT COUNT(*) FROM orders  
WHERE order_date BETWEEN '2024-07-01' AND '2024-07-10'  
AND status = 'pending'  
AND payment_status = 'paid';  
-- 结果：1,247条
```

3. 再次确认影响范围：

```
SELECT order_id, status, payment_status, total_amount  
FROM orders  
WHERE order_date BETWEEN '2024-07-01' AND '2024-07-10'  
AND status = 'pending'  
AND payment_status = 'paid'  
LIMIT 10;
```

4. 执行更新（小批量分批执行，避免大事务）：

```
UPDATE orders  
SET status = 'processing', updated_at = NOW()  
WHERE order_date BETWEEN '2024-07-01' AND '2024-07-10'  
AND status = 'pending'  
AND payment_status = 'paid'  
AND order_id <= 50000; -- 第一批
```

-- 确认无误后继续第二批...

5. 验证修复结果：

```
SELECT COUNT(*) FROM orders
WHERE order_date BETWEEN '2024-07-01' AND '2024-07-10'
AND status = 'pending'
AND payment_status = 'paid';
-- 结果：0条（修复成功）
```

6. 保留备份表一周后删除。

经验教训：

- 任何生产数据修改必须有备份和回滚方案；
- 大批量更新应分批执行，每批控制在1万行以内；
- 执行时间避开业务高峰（凌晨2-5点）；
- 双人复核（一人执行，一人审核SQL）。